



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Python Basics and Standard Library

Scope of variables

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Scope of a variable

- The **scope** of a variable refers to the **region or area in a program where a variable is recognized and can be accessed or modified.**



PROGRAMMING WITH PYTHON

Scope of a variable

Scope Type	Description	Example
L – Local	Variables defined inside a function.	Exists only within that function.
E – Enclosing	Variables in the outer (enclosing) function of a nested function.	Used in nested functions.
G – Global	Variables defined at the top level of a script or module.	Accessible anywhere unless shadowed.
B – Built-in	Names preassigned by Python (like len(), range, print).	Always available.



PROGRAMMING WITH PYTHON

Scope of a variable - Local

- A variable created **inside a function** belongs to the **local scope**.

```
def greet():  
    message = "Hello, World!"  # Local variable  
    print(message)
```

```
greet()
```

```
Hello, World!
```

```
print(message)
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[3], line 1  
----> 1 print(message)  
  
NameError: name 'message' is not defined
```



PROGRAMMING WITH PYTHON

Scope of a variable - Global

- A variable created **outside all functions** is a **global variable** and can be accessed anywhere (inside or outside functions).

```
x = 10 # Global variable
```

```
def display():  
    print("Inside function:", x)
```

```
display()
```

```
Inside function: 10
```

```
print("Outside function:", x)
```

```
Outside function: 10
```



PROGRAMMING WITH PYTHON

Scope of a variable - Global

- A variable created **outside all functions** is a **global variable** and can be accessed anywhere (inside or outside functions).

```
x = 10 # Global variable
```

```
def display():  
    print("Inside function:", x)
```

```
display()
```

```
Inside function: 10
```

```
print("Outside function:", x)
```

```
Outside function: 10
```



PROGRAMMING WITH PYTHON

Scope of a variable

- The scope of a variable determines the portion of the program where a particular identifier can be accessed.
 - Global variables
 - Local variables
- The statement `global VarName` tells Python that `VarName` is a global variable.
- Python stops searching the local namespace for the variable.



PROGRAMMING WITH PYTHON

Lifetime of a variable

- The period of time that a variable exists is called its **lifetime**.
- Local variables are automatically created (allocated memory) when a function is called, and destroyed (deallocated) when the function terminates.



PROGRAMMING WITH PYTHON

Lifetime of a variable

- Thus, the lifetime of a local variable is equal to the duration of its function's execution.
- Consequently, the values of local variables are not retained from one function call to the next.



PROGRAMMING WITH PYTHON

Lifetime of a variable

- Scope = The stage area where the variable is visible to the code.
- Lifetime = The time duration the variable is present in the code before getting terminated.



PROGRAMMING WITH PYTHON

Global Variables

- A **global variable** is a variable that is defined outside of any function definition.
- Such variables are said to have **global scope**.

```
max = 100 # global variable
```

```
def func1(count):  
    if count < max:  
        .
```

global variable max
accessed

```
def func2(count):  
    for i in range(1, max):  
        .
```

global variable max
accessed



PROGRAMMING WITH PYTHON

Difference between Scope and Lifetime

• Feature	Scope	Lifetime
Meaning	Area in code where variable is visible	Period in memory when variable exists
Nature	Spatial (where)	Temporal (when)
Controlled by	Where the variable is defined	When the block/function is executed



PROGRAMMING WITH PYTHON

Difference between Scope and Lifetime

• Feature	Scope	Lifetime
Ends when	Program leaves that code block	Variable’s memory is freed (e.g., function ends or object deleted)
Error type if violated	<code>NameError</code> (if accessed outside scope)	Variable may not exist in memory, leading to access failure



PROGRAMMING WITH PYTHON

Nested Functions

- A nested function is simply a function within another function, and is sometimes called an "**inner function**".
- Inner functions has direct access to variables and names defined in the enclosing function.



PROGRAMMING WITH PYTHON

Nested Functions

```
def outer_funcname():
```

```
    def inner_funcname():
```

```
        block
```

```
    inner_funcname()
```

```
outer_funcname()
```




PROGRAMMING WITH PYTHON

Nested Functions

```
def outer_func():  
    def inner_func():  
        print("Hello, Python!")  
    inner_func()  
outer_func()
```

Hello, Python!



PROGRAMMING WITH PYTHON

Nested Functions

- The major use of internal functions is their easiness to access variables and objects from an subscribed (“external”) function.
- The enclosing function provides a namespace available to the nested function



PROGRAMMING WITH PYTHON

Nested Functions

```
def outer_func(who):  
    def inner_func():  
        print(f"Hello, {who}")  
    inner_func()  
name=input("Enter the value")  
outer_func(name)
```

Enter the value Harry

Hello, Harry



PROGRAMMING WITH PYTHON

Nested Functions

- Scope of variables in Nested Functions
- Accessing the global variable from a function

```
def func1():  
    s = 'Python Program'  
    def func2():  
        s = 'Program with Python'  
        print(s)  
    func2()  
    print(s)  
func1()
```

Program with Python
Python Program



PROGRAMMING WITH PYTHON

Nested Functions

- Scope of variables in Nested Functions

```
def outer_function():  
    print("Inside Outer Function")  
    enclosing_var = "Enclosing Variable"  
  
    def inner_function():  
        local_var = "Local Variable"  
        print("Inside inner_function:")  
        print("Local Variable:", local_var)  
        print("Enclosing Variable:", enclosing_var)  
  
    inner_function()  
  
outer_function()
```

```
Inside Outer Function  
Inside inner_function:  
Local Variable: Local Variable  
Enclosing Variable: Enclosing Variable
```



PROGRAMMING WITH PYTHON

Nested Functions

- Scope of variables in Nested Functions

```
def outer_function():  
    print("Inside Outer Function")  
    enclosing_var = "Enclosing Variable"  
  
    def inner_function():  
        local_var = "Local Variable"  
        print("Inside inner_function:")  
        print("Local Variable:", local_var)  
        print("Enclosing Variable:", enclosing_var)  
  
    inner_function()  
    print("Local Variable:", local_var)  
  
outer_function()
```

Inside Outer Function
Inside inner_function:
Local Variable: Local Variable
Enclosing Variable: Enclosing Variable

Inside Outer Function
Inside inner_function:
Local Variable: Local Variable
Enclosing Variable: Enclosing Variable

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[9], line 14  
     11     inner_function()  
     12     print("Local Variable:", local_var)  
--> 14 outer_function()  
  
Cell In[9], line 12, in outer_function()  
      9     print("Enclosing Variable:", enclosing_var)  
     11     inner_function()  
--> 12     print("Local Variable:", local_var)  
  
NameError: name 'local_var' is not defined
```



PROGRAMMING WITH PYTHON

Nested Functions

```
def outerFunc():  
    sample_str = 'Outer Function'  
    def innerFunc():  
        sample_str = 'Inner Function'  
        print(sample_str)  
    innerFunc()  
    print(sample_str)  
outerFunc()
```

Inner Function

Outer Function



PROGRAMMING WITH PYTHON

Packing and Unpacking

- Packing
 - As its name suggests, it wraps all the arguments into a single variable and this function call gets into a tuple called args.
 - Can use any other name in place of args.
 - Example
 - `def func( *args )`



PROGRAMMING WITH PYTHON

Packing and Unpacking

- Unpacking
 - Unpacking is the term which refers to an operation list or tuple assigned to a single variable.
 - Example
 - $(a, b, c) = \text{List/tuple}$



PROGRAMMING WITH PYTHON

Packing and Unpacking

- Unpacking
 - Unpacking is the term which refers to an operation list or tuple assigned to a single variable.
 - Example
 - $(a, b, c) = \text{List/tuple}$



PROGRAMMING WITH PYTHON

Packing and Unpacking

```
# Packing example  
def pack_values(*args):  
    print("Packed values:", args)
```

```
# Packing values  
pack_values(1, 2, 3, 4, 5)
```



PROGRAMMING WITH PYTHON

Packing and Unpacking

Unpacking example

```
def unpack_values(packed_tuple):  
    a, b, c = packed_tuple  
    print("Unpacked values:", a, b, c)
```

Creating a tuple for unpacking

```
packed_tuple = (10, 20, 30)  
unpack_values(packed_tuple)
```



PROGRAMMING WITH PYTHON

Recap

- Scope of variables
- Life time of a variable
- Nested Functions
- Packing and Unpacking



THANK YOU

Lekha A

Computer Applications